PROYECTO FINAL · ANALÍTICA DE DATOS I · 2026-02

# Proyecto 3: Nivel de Generación Solar – Clasificación (Planta 1)

Clasificación del nivel de generación de una planta fotovoltaica real

| | |
|---|---|
| **Modalidad** | Un (1) equipo de 3 personas |
| **Peso en la nota final** | 30 % (Proyecto Final) |
| **Duración** | 4 semanas: del miércoles 28 de octubre al miércoles 25 de noviembre de 2026 |
| **Entrega de notebook, informe y diapositivas** | Martes 24 de noviembre de 2026, 11:59 p. m. |
| **Sustentación** | Miércoles 25 de noviembre de 2026 (semana 17) |
| **Uso de IA generativa** | Nivel 3 – Colaboración con IAG (máximo permitido). Ver la sección "Reglas de juego: uso de IA generativa". |
| **Datos** | Descargados por el grupo desde Kaggle (cuenta gratuita): https://www.kaggle.com/datasets/anikannal/solar-power-generation-data. Trabajo en Jupyter local o Google Colab. Usar solo los archivos de la Planta 1. |

## Contexto del Caso

Una **empresa de operación y mantenimiento (O&M) de plantas solares** quiere ofrecer sus servicios en Colombia y necesita herramientas analíticas para vigilar el desempeño de las plantas. Para desarrollarlas usará datos reales de la **Planta 1** de un conjunto publicado en Kaggle: la potencia de sus **22 inversores** medida cada 15 minutos durante 34 días (15 de mayo al 17 de junio de 2020) y un sensor de irradiación y temperatura en la planta.

**Problema:** la empresa programa sus tareas de mantenimiento en los periodos de generación baja y necesita anticipar, con 15 minutos de antelación, en qué nivel de generación estará la planta. Si el nivel real resulta más bajo que el anticipado, hay pérdidas que deben revisarse.

En este proyecto se trabaja con la **planta completa** (no con cada inversor): la potencia de la planta en cada instante, **AC_PLANTA_KW**, es la suma de la potencia AC de todos sus inversores. Trabajando solo con las horas de sol, deben resolver la siguiente tarea:

- **Clasificación:** clasificar el instante siguiente (t + 1), cada 15 minutos, en un **nivel de generación alto, medio o bajo** a partir de la información disponible hasta el instante t: las mediciones del sensor, la hora y los valores pasados de la potencia. Los tres niveles se definen con los **terciles de AC_PLANTA_KW calculados solo con el conjunto de entrenamiento**, usando la función

`crear_niveles` del Anexo B. Deben trabajar con tres niveles; los terciles son la definición por defecto y solo pueden usar umbrales fijos en kW si los justifican con la Tarea 1.

- **Alcance:** el modelo estima el nivel del instante siguiente (t + 1) con información disponible de instantes anteriores en t.

## Objetivos y Resultados Esperados

### Objetivo general

Construir y evaluar un modelo de clasificación que anticipe el nivel de generación (alto, medio o bajo) del instante siguiente en la Planta 1 a partir de las condiciones medidas, y explicar qué variables lo determinan.

### Objetivos específicos

- Documentar el contexto energético y operativo de la planta y formular dos o tres preguntas analíticas (Tarea 1).
- Construir un dataset limpio de la planta, con las decisiones de calidad de datos justificadas (Tarea 2).
- Comparar las líneas base (clase más frecuente y persistencia de la clase) con regresión logística, KNN y árbol de decisión, usando validación cronológica (Tarea 3).
- Interpretar los resultados, responder las preguntas analíticas y proponer cómo usaría la empresa el modelo (Tarea 4 y sustentación).

### Resultados esperados

Al final del proyecto el grupo debe presentar, en el notebook y en el informe:

- El dataset de la planta (AC_PLANTA_KW, variables del sensor y variables construidas) y un resumen de las decisiones de calidad de datos.
- Una tabla comparativa con accuracy, F1 macro y F1 por clase en el conjunto de prueba para todos los modelos, incluidas las líneas base.
- La matriz de confusión del mejor modelo y las reglas principales del árbol de decisión.
- La interpretación de los efectos de las variables y de la importancia de variables, conectada con la Tarea 1.
- La respuesta a las preguntas analíticas y una recomendación concreta para la empresa.

**No se exige un valor mínimo de las métricas.** Se evalúan la calidad de la preparación de los datos, la validación, la comparación contra las líneas base y sobre todo la interpretación de los resultados, según la rúbrica.

## Recursos Disponibles

| Recurso | Dónde encontrarlo | Para qué sirve |
|---|---|---|
| Datos | Kaggle (cuenta gratuita, botón Download): https://www.kaggle.com/datasets/anikannal/solar-power-generation-data | Usar solo Plant_1_Generation_Data.csv y Plant_1_Weather_Sensor_Data.csv |
| Diccionario de datos | Anexo A de este documento | Significado y unidades de cada columna |
| Ayudas de código probadas | Anexo B de este documento | Ubicar los datos, crear rezagos, crear la variable de clasificación y calcular la persistencia de la clase |
| Plantilla del informe | Plantilla IEEE del curso ("Formato presentación documentos") en la plataforma del curso | Formato obligatorio del informe |
| Rúbrica | Sección "Rúbrica de evaluación" de este documento | Criterios, pesos y evidencias que se calificarán |
| Acompañamiento | Sesiones 13 a 16: asesoría y revisión de avances | Resolver dudas antes de la entrega |

## Ruta del Proyecto (CRISP-DM)

El proyecto sigue las fases de la metodología CRISP-DM vista en la sesión 2. Cada tarea de este enunciado corresponde a una o más fases:

| Fase CRISP-DM | Tarea del proyecto | Pregunta que responde |
|---|---|---|
| Comprensión del negocio | Tarea 1: Contexto Energético y Operativo de la Planta | ¿Qué problema operativo tiene la empresa de O&M, qué factores físicos y de operación explican la generación de la planta y qué preguntas analíticas vamos a responder? |
| Comprensión de los datos | Tarea 2 (datos de Kaggle y EDA) | ¿Qué datos tenemos, qué tan completos son y qué patrones muestran? |
| Preparación de los datos | Tarea 2 (faltantes, atípicos, variables) | ¿Qué decisiones de limpieza y transformación tomamos y por qué? |
| Modelado | Tarea 3 | ¿Qué modelos usamos y cómo ajustamos sus hiperparámetros? |
| Evaluación | Tareas 3 y 4 | ¿El modelo supera la línea base y responde las preguntas del negocio? |
| Despliegue (propuesta) | Tarea 4 (conclusiones) y sustentación | ¿Cómo usaría la empresa el modelo y qué haría falta para ponerlo en operación? |

## Cronograma Sugerido (4 Semanas)

Las fechas de **entrega (martes 24 de noviembre)** y de **sustentación (miércoles 25 de noviembre)** son obligatorias. Las metas semanales son una guía; las revisiones de las sesiones son formativas y no se califican por separado.

| Sem. | Fechas | Qué deben hacer | Producto de la semana | En clase |
|---|---|---|---|---|
| 1 | 28 oct – 3 nov | Tarea 1 (contexto y preguntas analíticas) y descarga de los datos desde Kaggle, preparación del entorno y construcción de la serie de la planta | Introducción con fuentes y datos cargados | Sesión 13 (28 oct): atípicos e imputación |
| 2 | 4 – 10 nov | Tarea 2: calidad de datos, faltantes, atípicos y EDA | Dataset limpio y gráficos del EDA | Sesión 14 (4 nov): asesoría del proyecto |
| 3 | 11 – 17 nov | Tarea 3: modelos de clasificación/forecasting con validación | Tabla comparativa de modelos | Sesión 15 (11 nov): Parcial 3 y avances |
| 4 | 18 – 24 nov | Tarea 4: informe IEEE, limpieza del notebook y diapositivas | Entrega final (martes 24 nov) | Sesión 16 (18 nov): revisión de avances |
| — | 25 nov | Sustentación | Presentación y defensa | Semana 17 (miércoles 25 nov) |

## Reglas de Juego: Uso de IA Generativa

De acuerdo con los acuerdos sobre IAG del syllabus del curso y el Manifiesto sobre IA y educación de la Universidad Icesi, este proyecto se desarrolla como máximo en el **Nivel 3 – Colaboración con IAG**. Pueden apoyarse en la IAG para completar tareas y mejorar sus productos, pero llevando un registro de sus interacciones y demostrando que entienden y pueden modificar todo lo generado.

### Qué está permitido

- Pedir explicaciones de conceptos, de funciones de pandas y scikit-learn o de mensajes de error.
- Generar o depurar fragmentos de código (gráficos, limpieza, ajuste de modelos), siempre que el grupo lo revise, lo ejecute, lo entienda y pueda modificarlo.
- Mejorar la redacción y la ortografía del informe y de las diapositivas.
- Buscar fuentes para la Tarea 1, verificando cada una en el documento original antes de citarla.

### Qué no está permitido

- Entregar código, texto o figuras que ningún integrante pueda explicar o modificar.

- Delegar en la IAG las decisiones del proyecto y su sustentación (qué variables usar, cómo tratar faltantes y atípicos, qué modelo elegir) o la interpretación de resultados, la discusión y las conclusiones: esas son las evidencias de aprendizaje que se evalúan.
- Citar fuentes, cifras o resultados sin verificarlos: la IAG puede inventar referencias y datos.

### Registro y declaración (obligatorios)

- **Declaración en el informe:** un párrafo antes de las referencias con el nivel usado, las herramientas, para qué se usaron y cómo se verificó lo generado.

### Cómo se verifica

- En la sustentación, cualquier integrante puede ser llamado a explicar o modificar en vivo cualquier parte del notebook o del informe. La rúbrica (criterio 5, 30 %) ubica en nivel Developing la "dependencia significativa de herramientas de IA sin comprensión plena de lo generado".
- Entregar sin registro, con contenido que el grupo no puede explicar o con fuentes inventadas afecta la calificación de los criterios correspondientes y puede tratarse como falta a la integridad académica según el reglamento de la Universidad.

## Contexto Energético y Operativo de la Planta (Tarea 1)

Investiguen y documenten en la introducción del informe, con al menos **cinco fuentes confiables** citadas en formato IEEE (UPME, XM, CREG, IDEAM, artículos científicos):

- **Energía solar en Colombia:** ¿cuánta capacidad solar tiene el Sistema Interconectado Nacional y cómo ha crecido? ¿Qué promueven la Ley 1715 de 2014 y la Ley 2099 de 2021?
- **Física de la planta:** ¿cómo dependen la potencia de la irradiación y de la temperatura del módulo? ¿Qué hace un inversor y por qué la potencia AC es menor que la DC?
- **Operación y mantenimiento:** ¿qué causa pérdidas en una planta solar (temperatura, suciedad, sombras, desconexiones, saturación del inversor)? ¿Qué es el performance ratio?
- **Estado del arte:** ¿cómo se usa el machine learning para clasificar el nivel de generación de plantas solares y qué variables resultan más importantes?

### Preguntas analíticas

Con base en lo investigado, formulen **dos o tres preguntas analíticas** que su proyecto responderá con los datos, y retómenlas en la discusión. Ejemplo: "¿Qué condiciones de irradiación y temperatura separan un nivel de generación alto de uno medio?"

# Comprensión y Preparación de los Datos (Tarea 2)

## 2.1 Descarga y construcción del dataset de la planta

- **Descarga:** entren a la página del dataset en Kaggle (cuenta gratuita), usen el botón **Download** y guarden el ZIP (archive.zip). Kaggle se usa solo para descargar los datos; el trabajo se hace en **Jupyter en su equipo** (recomendado) o en **Google Colab**.
- **Local:** creen una carpeta del proyecto, pongan allí el notebook y el ZIP (o los CSV descomprimidos) e instalen las librerías con `pip install pandas scikit-learn matplotlib`. **Colab:** suban el ZIP con el panel de archivos (queda en /content) o léanlo desde Google Drive. En ambos casos la función `buscar_datos` del Anexo B encuentra los CSV y descomprime el ZIP si hace falta.
- Usen solo **Plant_1_Generation_Data.csv** (potencia por inversor) y **Plant_1_Weather_Sensor_Data.csv** (sensor de la planta).
- Sumen la potencia AC de los inversores en cada instante para obtener **AC_PLANTA_KW** y unan el resultado con el sensor por fecha-hora. Creen variables derivadas justificadas (por ejemplo, la hora del día en decimal).

## 2.2 Selección de variables

- Elijan y justifiquen los predictores con base en la Tarea 1. El sensor aporta tres variables: IRRADIATION, MODULE_TEMPERATURE y AMBIENT_TEMPERATURE; a ellas se suman las variables de calendario que construyan (por ejemplo, la hora). Como el modelo estima el instante t + 1, las variables del sensor solo pueden entrar con su valor en t o en instantes anteriores: su valor en t + 1 todavía no se conoce.
- **Valores pasados (rezagos):** construyan con la función `agregar_rezagos` del Anexo B los valores anteriores de AC_PLANTA_KW y de las variables del sensor (por ejemplo, de hace 15, 30 y 60 minutos). En la fila de cada instante, el objetivo es AC_PLANTA_KW y los predictores son esos rezagos (_lag1 corresponde al instante anterior). Calcúlenlos con la fecha y no con la posición de la fila, y antes de filtrar las horas de sol. Comparen modelos con distintos rezagos: ¿cuánto aporta el clima reciente por encima de la historia de la potencia?
- **Justifiquen lo que descartan:** la potencia DC y las energías acumuladas (DAILY_YIELD, TOTAL_YIELD) se calculan a partir de la misma potencia con la que se definen las clases; usarlas es fuga de información. Lo mismo ocurre con AC_PLANTA_KW y con las variables del sensor del mismo instante: solo sus valores pasados son válidos.

## 2.3 Faltantes, atípicos y calidad de datos

- **Registros ausentes:** calculen cuántos instantes de 15 minutos deberían existir en el periodo y cuántos faltan en cada archivo.
- **Instantes incompletos:** cuenten cuántos inversores reportan en cada instante. Si faltan inversores, la suma subestima la potencia de la planta: decidan si excluir esos instantes o corregirlos, y justifiquen.

- **Cero estructural:** de noche la potencia y la irradiación son cero; eso no es un faltante. Filtren las horas de sol y justifiquen el umbral de irradiación que usen.
- **Formatos de fecha:** no son iguales en los dos archivos. Revísenlos antes de unir.
- **Error de escala:** comparen DC_POWER con AC_POWER. ¿La relación es físicamente posible? Documenten el hallazgo, aunque la potencia DC no se use como predictor.
- **Atípicos:** identifíquenlos con una técnica estadística sencilla (rango intercuartílico o puntaje z, sesión 13) y decidan, caso por caso, si son errores de registro o eventos reales. No eliminen datos sin justificarlo.
- **Imputación:** usen técnicas sencillas y justificadas (mediana por grupo, interpolación en el tiempo). Si imputan, comparen la distribución antes y después.

### 2.4 Análisis exploratorio

- Histograma y boxplot de AC_PLANTA_KW: ¿cómo es su distribución?, ¿hay días atípicos (por ejemplo, nublados)?
- AC_PLANTA_KW vs. IRRADIATION (¿es lineal en todo el rango?), MODULE_TEMPERATURE vs. AMBIENT_TEMPERATURE, perfil diario promedio de potencia y matriz de correlación.
- Distribución de las tres clases en entrenamiento y en prueba: ¿se mantiene la proporción? ¿Por qué podría cambiar entre los primeros y los últimos días?

## Modelado y Evaluación (Tarea 3)

Vayan de lo simple a lo complejo y comparen siempre contra una línea base. Todo con scikit-learn, como en las sesiones 8 a 10.

### 3.1 Clasificación: nivel de generación

- **Línea base:** un modelo que siempre predice la clase más frecuente del entrenamiento (DummyClassifier). Reporten también la **persistencia de la clase**: el nivel de los 15 minutos anteriores, calculado con AC_PLANTA_KW_lag1 y los mismos cortes (código en el Anexo B).
- **Regresión Logística:** interpreten el efecto de las variables más influyentes sobre la probabilidad de cada clase.
- **KNN:** escalen las variables y elijan k con validación cruzada.
- **Árbol de Decisión:** elijan la profundidad máxima con validación cruzada y visualicen las primeras reglas: ¿tienen sentido para el negocio?
- **Métricas:** accuracy, precision, recall y F1 por clase, y matriz de confusión. Analicen entre qué clases se equivoca más el modelo y por qué.

### 3.2 Validación

- Dividan los datos **cronológicamente por días**: los primeros días (aprox. 80 %) para entrenamiento y los últimos (aprox. 20 %) para prueba. Expliquen por qué no conviene mezclar al azar registros consecutivos de 15 minutos.
- Ajusten hiperparámetros con **validación cruzada de 5 particiones para series de tiempo** (TimeSeriesSplit de scikit-learn) sobre el entrenamiento ordenado por fecha.
- El escalado y la imputación se ajustan **solo con el conjunto de entrenamiento** (usen Pipeline de scikit-learn). Los cortes de las clases también se calculan solo con entrenamiento (la función `crear_niveles` lo hace así).
- **Sobreajuste:** comparen la métrica principal en entrenamiento y en prueba para cada modelo y comenten si hay sobreajuste.
- **Interpretabilidad:** presenten la importancia de variables del árbol de decisión y relaciónenla con lo investigado en la Tarea 1.

***Opcional (no obligatorio):*** *Random Forest o XGBoost para comparar, o K-Means para agrupar los días según su perfil de generación (despejados vs. nublados) y discutir si los grupos tienen sentido de negocio.*

## Informe Final (Tarea 4)

Documento técnico de **máximo 5 páginas** en la plantilla IEEE del curso ("Formato presentación documentos"). No anexen el código al documento. Estructura:

- **Título, autores y resumen (150–250 palabras)** e índice de términos (cuatro palabras clave).
- **Introducción:** problema de negocio, contexto de la Tarea 1, objetivos y preguntas analíticas.
- **Materiales y métodos:** datos, variables seleccionadas, manejo de faltantes y atípicos, modelos, validación, métricas y versiones de las librerías.
- **Resultados:** tabla comparativa de todos los modelos (incluida la línea base) y las figuras principales, presentados sin interpretar.
- **Discusión:** respuesta a las preguntas analíticas, interpretación a la luz del negocio y limitaciones. Por ejemplo: "La mayoría de errores ocurre entre los niveles medio y alto en las horas de transición de la mañana, cuando la irradiación cambia rápidamente".
- **Conclusiones:** utilidad del modelo para la toma de decisiones, cómo lo usaría la empresa (despliegue) y mejoras concretas.
- **Declaración de uso de IAG:** un párrafo antes de las referencias con el nivel usado, las herramientas, para qué se usaron y cómo se verificó lo generado (si no usaron IAG, decláranlo).
- **Referencias** en formato IEEE.

## Sustentación (miércoles 25 de noviembre)

- **Duración:** 12 minutos de presentación y 8 minutos de preguntas por grupo. Todos los integrantes presentan.
- **Contenido:** problema y preguntas analíticas, decisiones de preparación de datos, modelos y resultados, respuesta a las preguntas y recomendación para la empresa.
- **Preguntas individuales:** el docente puede pedir a cualquier integrante que explique o modifique en vivo una parte del notebook. Los criterios 1 a 4 de la rúbrica se califican por grupo; el criterio 5 (sustentación, 30 %) se califica de forma individual.

## Evidencias que se Revisarán en este Proyecto

| Criterio | Qué se espera ver |
|---|---|
| **1. Claridad y Rigor en el Análisis** | Descarga y unión correcta con formatos de fecha distintos; construcción de la potencia de la planta; registros ausentes e instantes incompletos; ceros nocturnos; error de escala DC/AC; atípicos con IQR o z; distribución de clases en entrenamiento y prueba. |
| **2. Desarrollo y Ajuste de los Modelos** | Variable objetivo construida con crear_niveles y justificada; rezagos comparados con la persistencia de la clase; efecto de las variables en la regresión logística interpretado; k y profundidad justificados; reglas del árbol con sentido físico; AC_PLANTA_KW y variables con fuga excluidas; importancia de variables discutida. |
| **3. Validación y Métricas** | División cronológica por días y validación cruzada para series de tiempo; terciles solo con entrenamiento; comparación contra la clase mayoritaria; matriz de confusión y F1 por clase; comparación entrenamiento vs. prueba. |
| **4. Comunicación de Resultados** | Informe IEEE de máximo 5 páginas con tabla comparativa, matriz de confusión, reglas principales del árbol e importancia de variables, explicadas en el texto; preguntas analíticas respondidas. |
| **5. Sustentación Oral (Presentación)** | Explicar cómo se definieron las clases, por qué AC_PLANTA_KW no puede ser predictor, por qué la división es cronológica y defender o modificar el código sin apoyo de la IAG. |

## Entregables

**Cómo entregar:** un solo envío por grupo en la plataforma del curso, a más tardar el **martes 24 de noviembre de 2026 a las 11:59 p. m.**, en un archivo ZIP llamado **Proyecto3_Grupo_ApellidosMiembros.zip** con los tres archivos siguientes. No incluyan los CSV del dataset.

- **Notebook (.ipynb, Jupyter local o descargado de Google Colab):** documentado, ejecutado de principio a fin con las salidas visibles y con semilla aleatoria fija (random_state = 42).
- **Informe final:** PDF de máximo 5 páginas en la plantilla IEEE del curso.
- **Diapositivas** de la sustentación (PDF).

## Rúbrica de Evaluación

| Criterio | Peso | Outstanding (4.5 – 5.0) | Proficient (3.8 – 4.5) | Developing (3.0 – 3.8) | Beginning (0 – 3.0) |
|---|---|---|---|---|---|
| **1. Claridad y Rigor en el Análisis** | 17.5 % | El EDA está bien estructurado, con gráficos claros y justificaciones sólidas y profundas sobre cada decisión de preprocesamiento. | El EDA está completo y ordenado, con justificaciones adecuadas para las decisiones tomadas. | El EDA presenta cierta estructura, pero carece de justificaciones claras o consistencia en las decisiones. | El EDA es superficial, desorganizado o con ausencia de justificaciones relevantes. |
| **2. Desarrollo y Ajuste de los Modelos** | 17.5 % | Implementación e interpretación correcta de modelos; se interpretan correctamente los resultados, se discuten supuestos y se analizan las variables relevantes. | Modelos correctamente implementados; interpretación y análisis adecuados, aunque con menor profundidad. | Modelos funcionales, pero con errores de interpretación o sin discusión de supuestos/variables. | Modelos mal implementados o no interpretados; sin relación con los objetivos del taller. |
| **3. Validación y Métricas** | 17.5 % | Aplicación de validación cruzada o estrategias pertinentes; uso acertado de métricas y análisis crítico profundo. | Validación bien ejecutada y métricas adecuadas; el análisis crítico es correcto, aunque puede mejorar. | Uso básico de validación y métricas; análisis poco crítico o limitado. | Validación incorrecta o ausente; métricas mal utilizadas o mal interpretadas. |
| **4. Comunicación de Resultados** | 17.5 % | Informe claro, bien estructurado y argumentado; visualizaciones claras, relevantes y bien explicadas. | Informe claro y coherente; visualizaciones adecuadas, aunque podrían mejorar en diseño o explicación. | Informe con deficiencias de estructura o claridad; visualizaciones poco efectivas o incompletas. | Informe confuso o incompleto; visualizaciones ausentes o sin valor analítico. |
| **5. Sustentación Oral (Presentación)** | 30 % | Domina con seguridad los conceptos técnicos y metodológicos del trabajo; justifica cada decisión (preprocesamiento, selección de modelo, métricas) con argumentos propios y contextualizados; explica y defiende su código sin depender de notas, respondiendo con solvencia preguntas no anticipadas. | Explica con claridad la mayoría de los conceptos y decisiones del trabajo, aunque con menor profundidad en algunos puntos; justifica adecuadamente las decisiones metodológicas; explica su código de forma correcta, con apoyo ocasional en notas o documentación propia. | Presenta el trabajo con explicaciones superficiales o memorísticas; tiene dificultad para justificar decisiones metodológicas sin ayuda del evaluador; su explicación del código es parcial o evidencia dependencia significativa de herramientas de IA sin comprensión plena de lo generado. | No logra explicar los conceptos ni justificar las decisiones tomadas; no puede explicar o defender el código presentado; depende casi por completo de las indicaciones del evaluador para sostener cualquier parte de la presentación. |

## Anexo A: Diccionario de Datos (Planta 1)

Periodo: 15-may-2020 a 17-jun-2020, registros cada 15 minutos. El archivo de sensores contiene solo las seis columnas indicadas; AC_PLANTA_KW la construyen ustedes. Las variables marcadas como "Construida" no están explícitas en el conjunto de datos: debe construirlas el equipo, algunas con las ayudas de código ubicadas al final o con el código que consideren pertinente.

| Variable | Archivo | Descripción | Unidad |
|---|---|---|---|
| DATE_TIME | Ambos | Fecha y hora del registro (formato distinto en cada archivo) | — |
| PLANT_ID | Ambos | Identificador de la planta | — |
| SOURCE_KEY | Ambos | Generación: identificador del inversor (solo para sumar y contar). Sensores: identificador del sensor | — |
| DC_POWER | Generación | Potencia DC que entra al inversor | kW |
| AC_POWER | Generación | Potencia AC que entrega el inversor | kW |
| DAILY_YIELD / TOTAL_YIELD | Generación | Energía acumulada del inversor en el día / total | kWh |
| AMBIENT_TEMPERATURE | Sensores | Temperatura ambiente de la planta | °C |
| MODULE_TEMPERATURE | Sensores | Temperatura del módulo | °C |
| IRRADIATION | Sensores | Irradiación sobre el plano del módulo (promedio de 15 min) | kW/m² |
| **AC_PLANTA_KW** | Construida | Base para definir la clase: suma de AC_POWER de todos los inversores en el instante | kW |
| AC_PLANTA_KW_lag1 / _lag2 / _lag4 | Construida | Potencia de la planta 15, 30 y 60 minutos antes (predictores permitidos) | kW |
| Sensor: <variable>_lag1 / _lag2 / _lag4 | Construida | IRRADIATION, AMBIENT_TEMPERATURE y MODULE_TEMPERATURE 15, 30 y 60 minutos antes (predictores permitidos) | kW/m², °C |
| **nivel** | Construida | **Variable objetivo:** alto, medio o bajo según los terciles de AC_PLANTA_KW en entrenamiento | — |

# Anexo B: Ayudas de Código

Son solo funciones de apoyo y la línea base de persistencia: la construcción del dataset, la división de los datos y los modelos los programa cada grupo. Prepárense para comprender y explicar también su funcionamiento en la sustentación.

## 1. Ubicación de los datos descargados de Kaggle (local o Colab)

```
import glob, os, zipfile
def buscar_datos(archivo='Plant_1_Weather_Sensor_Data.csv', carpeta='.'):
    """Devuelve la carpeta donde están los CSV del dataset descargado de Kaggle.
    Antes de ejecutar: descarguen el ZIP (botón Download en Kaggle) y déjenlo, o los CSV,
    dentro de `carpeta`:
    - Local (Jupyter): la carpeta del proyecto donde está el notebook (carpeta='.').
    - Colab: súbanlo con el panel de archivos (/content, carpeta='.') o desde Google Drive
      (carpeta='/content/drive/MyDrive/<su_carpeta>').
    Si encuentra el ZIP sin descomprimir, lo descomprime en <carpeta>/datos_solar."""
    hallados = glob.glob(os.path.join(carpeta, '**', archivo), recursive=True)
    if hallados:
        return os.path.dirname(hallados[0])
    for ruta_zip in glob.glob(os.path.join(carpeta, '*.zip')):
        with zipfile.ZipFile(ruta_zip) as z:
            if any(n.endswith(archivo) for n in z.namelist()):
                destino = os.path.join(carpeta, 'datos_solar')
                z.extractall(destino)
                return buscar_datos(archivo, destino)
    raise FileNotFoundError(f'No se encontró {archivo}. Descarguen el ZIP desde Kaggle y déjenlo '
                            'en la carpeta del notebook, o indiquen la ruta con el argumento carpeta.')
ruta = buscar_datos()
print('Datos en:', ruta, '->', sorted(os.listdir(ruta)))
```

## 2. Función para agregar valores pasados (rezagos)

```
import numpy as np
import pandas as pd
def agregar_rezagos(df, columna, pasos=(1, 2, 4), freq='15min'):
    """Agrega a df columnas con valores pasados de `columna` (rezagos).
    pasos: intervalos hacia atrás (con freq='15min': 1 = 15 min, 2 = 30 min, 4 = 1 hora).
    El rezago se busca por fecha y no por posición de fila: si falta el registro
    anterior, el valor queda vacío (NaN) en lugar de tomar uno equivocado.
    Requiere una columna DATE_TIME sin fechas repetidas. Úsenla ANTES de filtrar las horas de sol."""
    serie = df.set_index('DATE_TIME')[columna]
    if not serie.index.is_unique:
        raise ValueError('Hay fechas repetidas: agreguen o eliminen duplicados primero.')
    for p in pasos:
        pasado = serie.shift(p, freq=freq)       # el valor de t queda asociado a t + p intervalos
        df[f'{columna}_lag{p}'] = df['DATE_TIME'].map(pasado)
    return df


# Uso: df = agregar_rezagos(df, 'NOMBRE_DE_LA_COLUMNA')   -> crea NOMBRE_lag1, NOMBRE_lag2, NOMBRE_lag4
```

## 3. Función para crear la variable de clasificación

```
def crear_niveles(train, test, columna, n_niveles=3, etiquetas=None, umbrales=None, nombre='nivel'):
    """Crea la variable objetivo de clasificación a partir de una columna continua.
    - umbrales=None: cortes por cuantiles de TRAIN (n_niveles grupos de igual tamaño).
    - umbrales=[a, b, ...]: cortes fijos en las unidades de la columna (n_niveles = len(umbrales) + 1).
    - etiquetas: nombres de las clases de menor a mayor (por defecto bajo/medio/alto).
    Los cortes se calculan solo con train y se aplican igual a test.
    Devuelve copias de train y test con la nueva columna, y la lista de cortes."""
    if umbrales is None:
        cortes = [float(train[columna].quantile(i / n_niveles)) for i in range(1, n_niveles)]
```

```
    else:
        cortes = sorted(float(u) for u in umbrales)
        n_niveles = len(cortes) + 1
    if len(set(cortes)) != len(cortes):
        raise ValueError(f'Cortes repetidos {cortes}: usen menos niveles o umbrales fijos.')
    if etiquetas is None:
        etiquetas = {2: ['bajo', 'alto'], 3: ['bajo', 'medio', 'alto']}.get(
            n_niveles, [f'nivel_{i + 1}' for i in range(n_niveles)])
    if len(etiquetas) != n_niveles:
        raise ValueError(f'Se necesitan {n_niveles} etiquetas y se dieron {len(etiquetas)}.')
    limites = [-np.inf] + cortes + [np.inf]
    train, test = train.copy(), test.copy()
    for d in (train, test):
        d[nombre] = pd.cut(d[columna], bins=limites, labels=etiquetas)   # (a, b]: el corte va al nivel
inferior
    print('Cortes (calculados con train):', [round(c, 1) for c in cortes])
    print('Proporción por clase en train:', train[nombre].value_counts(normalize=True).round(2).to_dict())
    print('Proporción por clase en test: ', test[nombre].value_counts(normalize=True).round(2).to_dict())
    return train, test, cortes


# Uso: train, test, cortes = crear_niveles(train, test, 'AC_PLANTA_KW')                -> terciles
#      train, test, cortes = crear_niveles(train, test, 'AC_PLANTA_KW', n_niveles=2)    -> bajo / alto
```

## 4. Línea base de persistencia de la clase

```
# Línea base de persistencia de la clase: el nivel de la planta de los 15 minutos anteriores.
# Requiere `test` con las columnas nivel (crear_niveles) y AC_PLANTA_KW_lag1 (agregar_rezagos),
# y la lista `cortes` que devolvió crear_niveles.
from sklearn.metrics import classification_report


te = test.dropna(subset=['AC_PLANTA_KW_lag1'])
nivel_anterior = pd.cut(te['AC_PLANTA_KW_lag1'], bins=[-np.inf] + cortes + [np.inf],
                        labels=te['nivel'].cat.categories)
print(classification_report(te['nivel'], nivel_anterior))
```